

EXAM

Buntu Lwazi Sekgobela
ST10459712 ADDB6311

Github repository for questions 1 -5

<https://github.com/Buntu87/Preload.git>

Question 1

```
SELECT c.client_fname,  
       c.client_sname,  
       t.trail_name,  
       te.event_date  
FROM HikingAdventureMangementSystemdb.TRAIL_EVENT te  
JOIN  HikingAdventureMangementSystemdb.CLIENT c  
      ON te.client_id = c.client_id  
JOIN  HikingAdventureMangementSystemdb.TRAIL t  
      ON te.trail_id = t.trail_id  
ORDER BY te.event_date;
```

client_fname	client_sname	event_date
Nathan	Daniels	2025-07-05
Zara	Smith	2025-07-06
Jade	Pillay	2025-07-08
Sipho	Nkosi	2025-07-09
Hannah	Jacobs	2025-07-11
Tahir	Essop	2025-07-12
Nathan	Daniels	2025-07-15
Amelia	Jones	2025-07-17
Riley	Howard	2025-07-18

Question 2

```
SET SERVEROUTPUT ON;  
  
BEGIN  
  FOR rec IN (  
    SELECT t.trail_name AS event_name,  
           te.event_date AS allocated_date  
    FROM trail_event te  
    JOIN trail t
```

```

        ON te.trail_id = t.trail_id
WHERE LOWER(t.trail_name) LIKE '%dive%'

        AND te.participants >= 10
ORDER BY te.event_date
) LOOP

DBMS_OUTPUT.PUT_LINE(
    rec.event_name || ' ' || TO_CHAR(rec.allocated_date, 'YYYY-MM-DD')
);

END LOOP;

END;

/

```

event_name	allocated_date
Mountain Ridge	2025-07-05
Canyon Trek	2025-07-08
Highland Pass	2025-07-12
Sunset Hike	2025-07-15

Question 3

```

CREATE OR REPLACE VIEW "Trail Event_View" AS

SELECT te.guide_id,
       te.client_id,
       c.client_address,
       t.trail_duration
FROM trail_event te
JOIN client c
    ON te.client_id = c.client_id
JOIN trail t
    ON te.trail_id = t.trail_id
WHERE te.event_date BETWEEN DATE '2025-07-09' AND DATE '2025-07-23';

```

guide_id	client_id	client_address	trail_duration
202	CL105	31 Forest Lane	1 hour
205	CL107	96 Pine Road	2 hours
205	CL106	77 Oak Avenue	1 hour
205	CL101	14 Willow Street	3 hours
201	CL108	55 Riverbank	2 hours
202	CL109	12 Clover Street	1 hour

Question 4

CREATE OR REPLACE PROCEDURE Client_Details (

 p_client_id IN VARCHAR2,

 p_trail_date IN DATE

) AS

 v_client_fname VARCHAR2(100);

 v_client_sname VARCHAR2(100);

 v_trail_name VARCHAR2(100);

 v_found BOOLEAN := FALSE;

CURSOR c_details IS

 SELECT c.client_fname,

 c.client_sname,

 t.trail_name

 FROM trail_event te

 JOIN client c ON te.client_id = c.client_id

 JOIN trail t ON te.trail_id = t.trail_id

 WHERE te.client_id = p_client_id

 AND TRUNC(te.event_date) = TRUNC(p_trail_date);

BEGIN

 OPEN c_details;

LOOP

```

    FETCH c_details INTO v_client_fname, v_client_sname, v_trail_name;

    EXIT WHEN c_details%NOTFOUND;

    v_found := TRUE;

    DBMS_OUTPUT.PUT_LINE('Customer: ' || v_client_fname || ' ' || v_client_sname);

    DBMS_OUTPUT.PUT_LINE('Trail Name: ' || v_trail_name);

    DBMS_OUTPUT.PUT_LINE('---');

END LOOP;

CLOSE c_details;

IF NOT v_found THEN

    DBMS_OUTPUT.PUT_LINE('No client found for Client ID: ' || p_client_id || ' on date: ' ||
    TO_CHAR(p_trail_date, 'YYYY-MM-DD'));

END IF;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('ERROR: No data found for the specified criteria.');
```

WHEN OTHERS THEN

```

        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

END Client_Details;

/

SET SERVEROUTPUT ON;

-- Example 1: Client CL101 on 2025-07-05 (exists in preload data)
EXEC Client_Details('CL101', DATE '2025-07-05');
```

-- Example 2: Client CL105 on 2025-07-09 (exists in preload data)

```

EXEC Client_Details('CL105', DATE '2025-07-09');
```

-- Example 3: Client CL108 on 2025-07-17 (exists in preload data)

EXEC Client_Details('CL108', DATE '2025-07-17');

-- Example 4: Non-existent client (exception handling)

EXEC Client_Details('CL999', DATE '2025-07-05');

Expected output

Example 1 (CL101 on 2025-07-05):

```
Customer: Nathan Daniels
Trail Name: Eagle Point Trek
---
```

Example 2 (CL105 on 2025-07-09):

```
Customer: Sipho Nkosi
Trail Name: Forest Walk
---
```

Example 3 (CL108 on 2025-07-17):

```
Customer: Amelia Jones
Trail Name: Canyon Trek
---
```

Question 5

CREATE OR REPLACE FUNCTION fnClientReport (

 p_client_id IN VARCHAR2

) RETURN SYS_REFCURSOR AS

 v_cursor SYS_REFCURSOR;

 v_count NUMBER;

BEGIN

 -- Check if client exists

 SELECT COUNT(*)

 INTO v_count

 FROM client

```
WHERE client_id = p_client_id;
```

```
IF v_count = 0 THEN
```

```
    RAISE NO_DATA_FOUND;
```

```
END IF;
```

```
-- Open cursor with trail details
```

```
OPEN v_cursor FOR
```

```
    SELECT c.client_id AS customer_id,
```

```
           t.trail_id,
```

```
           te.event_date AS trail_date,
```

```
           t.trail_cost AS total_trail_cost
```

```
FROM trail_event te
```

```
JOIN client c ON te.client_id = c.client_id
```

```
JOIN trail t ON te.trail_id = t.trail_id
```

```
WHERE c.client_id = p_client_id
```

```
ORDER BY te.event_date;
```

```
RETURN v_cursor;
```

```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('ERROR: Client ID ' || p_client_id || ' not found.');
```

```
    RETURN NULL;
```

```
WHEN OTHERS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
```

```
    RETURN NULL;
```

```
END fnClientReport;
```

```
/
```

```

=== Trail Report for Client CL101 ===
Customer ID: CL101 | Trail ID: 658 | Trail Date: 2025-07-05 | Total Cost: $600
Customer ID: CL101 | Trail ID: 654 | Trail Date: 2025-07-15 | Total Cost: $400

=== Trail Report for Client CL105 ===
Customer ID: CL105 | Trail ID: 651 | Trail Date: 2025-07-09 | Total Cost: $150

=== Trail Report for Client CL999 ===
ERROR: Client ID CL999 not found.

```

Question 6

PL/SQL scalar data types store single, atomic values with no internal components. They hold data such as individual numbers, strings, dates, or logical flags. According to official [Oracle PL/SQL Documentation](#), scalar data types are grouped into four primary families: **Numeric**, **Character**, **Datetime**, and **Boolean**. [\[1, 2, 3\]](#)

1. Numeric Data Types

These types are explicitly designed to store mathematical numbers, allowing you to perform calculations like addition and multiplication. [\[1, 2, 3\]](#)

- **NUMBER:** Stores fixed-point or floating-point numbers with absolute precision. It supports up to 38 digits of precision.
- **PLS_INTEGER / BINARY_INTEGER:** Stores signed integers within the range of -2,147,483,648 to 2,147,483,647. It uses hardware arithmetic, making computations faster than NUMBER. [\[1, 2, 3, 4\]](#)

sql

DECLARE

v_salary NUMBER(8, 2) := 55000.50; -- Precision of 8 digits, 2 decimal places

v_quantity PLS_INTEGER := 150; -- Faster integer processing

BEGIN

NULL;

END;

/

2. Character Data Types

Character types are used for storing alphanumeric text, words, or free-form strings. [\[1\]](#)

- **VARCHAR2:** Stores variable-length text strings up to a maximum size of 32,767 bytes in PL/SQL blocks. It only utilizes space for the text actually stored. [\[1, 2, 3, 4, 5\]](#)

- CHAR: Stores fixed-length text strings up to 32,767 bytes. If the assigned text is shorter than the declared length, Oracle blank-pads the remaining space. [[1](#), [2](#), [3](#), [4](#), [5](#)]

sql

DECLARE

v_emp_name VARCHAR2(50) := 'Alice Smith'; -- Only uses 11 bytes of space

v_state_cd CHAR(2) := 'CA'; -- Always fills exactly 2 bytes

BEGIN

NULL;

END;

/

3. Datetime and Interval Data Types

These types are used to track specific chronological milestones or time durations. [[1](#), [2](#)]

- DATE: Stores calendar dates along with the specific hour, minute, and second.
- TIMESTAMP: Extends the core DATE type by supporting fractional seconds precision.
- INTERVAL DAY TO SECOND: Stores an absolute duration of elapsed time in days, hours, minutes, and seconds. [[1](#), [2](#), [3](#), [4](#), [5](#)]

sql

DECLARE

v_hire_date DATE := SYSDATE;

v_log_time TIMESTAMP(6) := SYSTIMESTAMP; -- Includes microsecond accuracy

v_grace_pd INTERVAL DAY TO SECOND := '05 02:00:00'; -- 5 days and 2 hours

BEGIN

NULL;

END;

/

4. Boolean Data Type

The BOOLEAN type is an exclusive PL/SQL-only data type. It cannot be directly stored inside Oracle database table columns. [[1](#), [2](#), [3](#), [4](#)]

- BOOLEAN: Evaluates exclusively to TRUE, FALSE, or NULL (representing an unassigned or unknown state). It is used to control logical program flow. [[1](#), [2](#)]

sql

DECLARE

```

v_is_eligible BOOLEAN := TRUE;

v_has_passed BOOLEAN; -- Defaults to NULL if left unassigned

BEGIN

  IF v_is_eligible THEN

    DBMS_OUTPUT.PUT_LINE('Access Granted');

  END IF;

END;

/

```

Summary of PL/SQL Scalar Families

Data Type Family [1, 2, 3, 4, 5]	Common Key Types	Best Used For
Numeric	NUMBER, PLS_INTEGER	Financial calculations, primary keys, and loop counters.
Character	VARCHAR2, CHAR	Names, descriptive text fields, and static status codes.
Datetime	DATE, TIMESTAMP	Transaction timestamps, creation dates, and log timing.
Boolean	BOOLEAN	Conditional statements and business logic flag tracking.

Question 7

Exception handling is a programming construct that intercepts runtime errors or anomalous events. It pauses execution, redirects flow to a controlled recovery or logging block, and prevents catastrophic system crashes. It improves system reliability and handles errors gracefully. [\[1, 2, 3, 4, 5, 6\]](#)

The following examples of predefined system exceptions justify the necessity of this mechanism:

1. ArithmeticException

- **Definition:** Triggered when an illegal mathematical operation is attempted, most commonly dividing by zero.
- **Justification:** Without exception handling, a System.DivideByZeroException (or similar language equivalent) halts the CPU's processing and crashes the application. Handling it allows the program to warn the user, prompt for a non-zero input, and continue functioning. [\[1, 2, 3, 4, 5\]](#)

2. NullPointerException

- **Definition:** Occurs when attempting to access a method, property, or value on an uninitialized (null) object or variable reference.
- **Justification:** Memory references can be unpredictable. Catching this exception intercepts a sudden program exit and redirects the flow to instantiate the object or use a safe fallback value, thereby protecting stability. [[1](#), [2](#), [3](#), [4](#), [5](#)]

3. IndexOutOfBoundsException

- **Definition:** Triggered when a program attempts to access an index that is less than zero or exceeds the size of an array, list, or string. [[1](#), [2](#)]
- **Justification:** Memory violations risk exposing private system buffers. By intercepting this exception, the program can safely restrict out-of-range inputs, preventing unauthorized memory reads and application faults. []

4. FileNotFoundException

- **Definition:** Thrown when a program attempts to read or write to a file system path that does not exist or lacks access permissions.
- **Justification:** File states are highly vulnerable to external changes beyond the software's control. Exception handling is essential to catch this event, avoiding crashed processes and instead allowing the system to create a new file or log a missing-configuration warning. [[1](#), [2](#), [3](#), [4](#), [5](#)]

5. OutOfMemoryException

- **Definition:** Thrown by the system runtime or garbage collector when the host environment runs out of usable RAM.
- **Justification:** Rather than having the operating system forcefully kill the application with no cleanup, catching this exception gives the software a final window to safely dump data, close active database connections, and terminate cleanly. [[1](#), [2](#), [3](#)]

Question 8

Securing an Oracle database running PL/SQL requires a layered approach across native database tools, programming practices, and enterprise security platforms. [[1](#), [2](#)]

Here is the breakdown of the essential tools and platforms to secure your PL/SQL environment.

Native Oracle Database Tools

- **Virtual Private Database (VPD):** Enforces row-level security dynamically.
- **Oracle Real Application Security (RAS):** Manages end-to-end application user sessions.
- **Oracle Data Redaction:** Masks sensitive data on the fly.
- **Oracle Transparent Data Encryption (TDE):** Encrypts data at rest automatically.
- **Unified Auditing:** Tracks PL/SQL execution and user actions. [[1](#), [2](#), [3](#), [4](#)]

PL/SQL Code Protection Tools

- **PL/SQL Hierarchical Profiler:** Identifies unauthorized code execution paths.

- **PL/Scope:** Analyzes PL/SQL source code for vulnerabilities.
- **DBMS_CRYPTO Package:** Provides native data encryption within code.
- **DBMS_ASSERT Package:** Sanitizes inputs to prevent SQL injection.
- **PL/SQL Wrapper (wrap):** Obfuscates source code to hide logic. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Enterprise Security Platforms

- **Oracle Key Vault:** Manages encryption keys and wallets centrally.
- **Oracle Audit Vault and Database Firewall:** Blocks SQL injection attacks remotely.
- **Oracle Data Safe:** Assesses security configurations and user risks.
- **Imperva Sonar / Guardium:** Monitors external database activity in real-time.
- **CyberArk / BeyondTrust:** Manages privileged database accounts and passwords. [[1](#), [2](#), [3](#), [4](#), [5](#)]

PL/SQL Secure Coding Best Practices

- **Bind Variables:** Use instead of dynamic string concatenation.
- **Definer vs Invoker Rights:** Use AUTHID CURRENT_USER by default.
- **Principle of Least Privilege:** Grant EXECUTE only to specific users.
- **Package Encapsulation:** Hide underlying tables behind PL/SQL APIs.
- **Grants:** Avoid granting privileges to the PUBLIC role. [[1](#), [2](#), [3](#), [4](#), [5](#)]